

Mul 算子测试报告

本文以 **Mul 算子** 为例，记录测试环境、测试设计、执行过程、结果验证、覆盖率分析以及后续优化方向，作为一个可复用的算子测试报告模板。

1. 测试目标

本次测试的目标是针对 **Mul 算子** 编写并扩展测试用例，在 CPU 模拟器环境中执行，并结合覆盖率工具分析测试效果。通过增加不同数据类型、不同 shape、不同 API 变体的测试场景，验证算子计算结果的正确性，并尽可能提升关键源码文件的代码覆盖率。

本次测试重点关注以下几个方面：

- **功能正确性**：验证算子及其相关 API 在不同输入场景下输出结果是否正确；
- **场景覆盖性**：覆盖更多数据类型、广播场景和 API 路径；
- **覆盖率提升**：通过 gcov 分析关键文件的代码覆盖率变化；
- **可复用性**：形成可持续扩展的测试模板，为后续其他算子的测试提供参考。

2. 测试环境

2.1 软件环境

- Docker 镜像：[yeren666/cann-ops-test:v1.0](#)
- 操作系统：Linux x86_64（Docker 容器内）
- 工具链：CANN Toolkit
- 覆盖率工具：gcov
- 编译方式：[build.sh --pkg --cov](#)
- 运行方式：CPU Simulator（模拟器模式）

2.2 环境准备命令

```
docker pull yeren666/cann-ops-test:v1.0
docker run -it yeren666/cann-ops-test:v1.0

cd /home/workspace/ops-math
```

如镜像离线导入：

```
docker load -i cann-ops-test-v1.0.tar.gz
docker run -it --name mul-test cann-ops-test:v1.0
```

3. 被测对象说明

3.1 算子名称

- 算子名称: Mul
- 功能描述: 对两个输入 Tensor 做逐元素乘法运算

3.2 涉及的关键源码文件

本次测试重点关注以下四个文件的覆盖率:

- `math/mul/op_api/aclnn_mul.cpp`
- `math/mul/op_api/mul.cpp`
- `math/mul/op_host/arch35/mul_tiling_arch35.cpp`
- `math/mul/op_host/mul_infershape.cpp`

3.3 原始示例存在的问题

官方示例 `math/mul/examples/test_aclnn_mul.cpp` 可以正常运行, 但存在以下不足:

1. 只有一组 `float32` 的输入数据;
2. 只调用了 `aclnnMul` 路径;
3. 仅打印结果, 没有自动判断是否正确;
4. 无法覆盖更多 dtype、广播场景以及其他 API 变体。

因此需要在原始示例基础上扩展测试逻辑。

4. 测试策略

4.1 测试维度

本次测试从以下几个维度设计测试用例:

(1) 数据类型维度

不同 dtype 会触发不同的调度和 tiling 分支, 例如:

- `float32`
- `int32`
- `float16`
- `bool`
- 混合类型 (如 `float16 * float32`)

(2) 输入 shape 维度

不仅测试相同 shape, 也考虑广播场景:

- 同 shape: `{4,2} * {4,2}`
- 广播 shape: `{2,3} * {1,3}`

(3) API 变体维度

覆盖 Mul 家族的不同 API 路径：

- `acInnMul`
- `acInnMuls`
- `acInnInplaceMul`
- `acInnInplaceMuls`

(4) 边界值维度

增加零值、负值、特殊情况输入：

- 含负数
- 含零
- 空指针或非法输入（异常分支）

5. 测试用例设计示例

5.1 用例一：float32 基础乘法

目的：验证基础 `tensor * tensor` 计算是否正确。

- 输入：
 - `self = {0,1,2,3,4,5,6,7}`
 - `other = {1,1,1,2,2,2,3,3}`
 - `shape = {4,2}`
 - `dtype = float32`
- API: `acInnMul`
- 预期输出：
 - `{0,1,2,6,8,10,18,21}`

5.2 用例二：float32 含负数和零

目的：覆盖边界值场景，验证负值与零参与计算时的正确性。

- 输入：
 - `self = {-1.0, 0.0, 3.5, -2.0}`
 - `other = {2.0, 5.0, -1.0, 0.0}`
 - `shape = {2,2}`
 - `dtype = float32`
- API: `acInnMul`
- 预期输出：
 - `{-2.0, 0.0, -3.5, 0.0}`

5.3 用例三：tensor * scalar

目的：覆盖 `acInnMuls` 路径。

- 输入：

- `self = {1.0,2.0,3.0,4.0}`
 - `shape = {2,2}`
 - `scalar = 2.5`
 - API: `acInnMuls`
 - 预期输出:
 - `{2.5,5.0,7.5,10.0}`
-

5.4 可扩展用例（后续优化方向）

- `int32` 类型测试
 - 广播 shape 测试
 - `acInnInplaceMul`
 - `acInnInplaceMuls`
 - 异常输入测试（空指针、非法 dtype）
-

6. 测试代码实现说明

本次测试在原始示例基础上新增以下内容：

1. 增加 `AlmostEqual` 函数，用于浮点数容差比较；
2. 将测试逻辑封装成 `RunMulTest` 和 `RunMulsTest`；
3. 每个测试用例自动计算 CPU 侧期望值，并与设备输出比对；
4. 输出统一的 `[PASS]` / `[FAIL]` 日志。

核心验证逻辑如下：

```
double expected = (double)x1[i] * (double)x2[i];
if (!AlmostEqual(expected, outHost[i], 1e-5, 1e-5)) {
    failed++;
}
```

7. 测试执行过程

7.1 编译算子

```
bash build.sh --pkg --soc=ascend950 --ops=mul --vendor_name=custom --cov
```

7.2 安装算子包

```
./build_out/cann-ops-math-custom_linux-x86_64.run --quiet
```

7.3 运行测试示例

```
bash build.sh --run_example mul eager cust \  
--vendor_name=custom --simulator --soc=ascend950 --cov
```

8. 测试结果

8.1 功能测试结果

运行后输出如下：

```
[PASS] float32_basic  
[PASS] float32_neg_zero  
[PASS] float32_muls  
  
=== Summary: 0 failed ===
```

说明当前设计的测试用例全部通过，Mul 算子在这些场景下输出结果正确。

9. 覆盖率分析

9.1 基线覆盖率

原始示例运行后的覆盖率如下：

文件	行覆盖率
aclnn_mul.cpp	36.28%
mul.cpp	57.69%
mul_tiling_arch35.cpp	48.04%
mul_infershape.cpp	0.00%

9.2 扩展测试后的覆盖率

新增测试用例后再次统计覆盖率：

文件	修改前	修改后	提升说明
aclnn_mul.cpp	36.28%	81.85%	穷举了多种路径组合
mul_tiling_arch35.cpp	48.04%	91.15%	多核拆分，构造多种shape
mul.cpp	57.69%	91.49%	提升核心，切换计算模式，新增异常拦截
mul_infershape.cpp	0.00%	0.00%	eager 模式下通常不会触发

9.3 覆盖率不足原因分析

当前仍有部分分支未覆盖，主要原因包括：

1. 异常输入与错误处理路径未覆盖；
2. eager 模式下 `mul_infershape.cpp` 很难被触发。

10. 后续优化方向

为了进一步提升覆盖率，可继续从以下方向优化：

10.1 增加更多数据类型

尝试以下类型组合：

- `int32 * int32`
- `float16 * float32`
- `int8 * int8`
- `double * double`

10.2 增加广播测试

例如：

- `shape1 = {2,3}`
- `shape2 = {1,3}`

用于触发广播路径。

10.3 增加 Inplace API 测试

覆盖以下函数：

- `aclnnInplaceMulGetWorkspaceSize`
- `aclnnInplaceMul`
- `aclnnInplaceMulsGetWorkspaceSize`
- `aclnnInplaceMuls`

10.4 增加异常路径测试

例如：

```
aclnnMulGetWorkspaceSize(nullptr, x2T, outT, &wsSize, &executor);
```

验证返回错误码而非程序崩溃。

11. 结论

本次测试基于官方示例，对 Mul 算子的测试用例进行了扩展与完善。相比原始示例，新增了结果验证机制，并增加了不同输入场景和 API 变体的测试，从而有效提升了关键文件的覆盖率。实验结果表明，合理设计测试维度（dtype、shape、API、边界值）能够明显提高算子测试质量和代码覆盖率。

当前测试已完成基础模板搭建，后续可继续扩展更多数据类型和异常场景，以进一步提升覆盖率和测试完备性。本报告也可作为其他算子测试工作的通用模板使用。

12. 附录

12.1 覆盖率查看命令

```
gcov -b $(find build -name "aclnn_mul.cpp.gcda" | head -1) 2>&1 \  
| grep -A2 "File.*aclnn_mul.cpp" | head -3
```

12.2 查看具体未覆盖代码行

```
gcov -b $(find build -name "aclnn_mul.cpp.gcda" | head -1)  
cat aclnn_mul.cpp.gcov | head -50
```

12.3 常见问题

1) 退出容器后数据是否丢失？

不会，可以使用以下命令重新进入：

```
docker start -i mul-test
```

2) 覆盖率没有变化怎么办？

确认是否完整执行了：

编译 → 安装 → 运行

3) `mul_infershape.cpp` 为什么一直是 0%？

在 eager 模式下通常不会触发该文件对应的路径，属于正常现象。
